Projet final de mise en situation professionnelle reconstituée

1 Objectifs pédagogiques et compétences visées

Cette mise en situation professionnelle reconstituée vise à évaluer la capacité du candidat à **concevoir, implémenter, expliquer et déployer des solutions d'intelligence artificielle dans un contexte réaliste**, en mobilisant des compétences issues des modules :

- **M32 Machine Learning II**
  - Concevoir, implémenter et documenter un pipeline de prévision de séries temporelles, incluant notamment des approches modernes : transformers, modèles de fondation.
  - Justifier les choix architecturaux et comparer différentes approches.
  - Expliquer et interpréter les résultats obtenus à l'aide d'un modèle ML.
- **M33 Deep Learning II**
  - Concevoir, implémenter et documenter un pipeline de classification d'images basé sur des réseaux de neurones profonds.
  - Concevoir, implémenter et documenter un pipeline de classification de textes en utilisant des méthodes NLP traditionnelles, ainsi que des réseaux de neurones profonds, notamment les LSTM.
  - Concevoir et implémenter une fusion multimodale combinant plusieurs types de données (images, texte, capteurs, etc.) afin d'améliorer la performance des modèles d'IA.
- **M34 MLOps**
  - Développer un code Python structuré en séparant les différentes phases d'un pipeline ML, telles que le prétraitement des données, l'entraînement et la génération des prédictions à l'aide d'un modèle d'apprentissage automatique.
  - Gérer les expérimentations et le versionnement des modèles d'IA (MLflow).
  - Mettre en place une orchestration (Airflow).
  - Déployer un modèle via une API (FastAPI).
  - Implémenter un monitoring du modèle.
- **M27 DevOps**
  - Créer une image docker pour empaqueter le code Python de l'API incluant des fichiers nécessaires (tels que modèles ML sérialisés par exemple).
  - Ajouter un logging pertinent pour chaque handler de endpoint.
  - Ajouter un endpoint pour connaitre la version logicielle actuelle.
  - Concevoir, implémenter et exécuter des tests automatisés des endpoints d'une API.
  - Définir un pipeline basé sur GitHub Actions qui crée une image docker, l'envoie vers ghcr.io et exécute des tests de l'API.

2 Format d'évaluation

Chaque étudiant travaillera **individuellement** sur deux sous-projets :

- Un premier portant sur l'analyse d'images et de textes, ainsi que leur fusion.
- Un second consacré à la conception et l'implémentation d'un pipeline complet de prévision de séries temporelles. **Le pipeline de déploiement devra être mis en place uniquement pour ce modèle**.

L'évaluation prendra la forme d'une **soutenance orale individuelle** durant laquelle le candidat présentera et défendra les réflexions qu'il aura menées et les solutions qu'il aura apportées. Les livrables serviront de base au jury pour questionner l'apprenant sur des points particuliers.

CALENDRIER :

La date limite de restitution des livrables est le mardi **07/04/2025 à 00h**.

Les soutenances seront organisées dans la période **du 07/04/2025 au 10/04/2025**.

L'organisation d'une soutenance est la suivante :

- 15 minutes pour l'exposé et la démonstration,
- 20 minutes de questions-réponses.

Le respect du temps imparti fait partie intégrante de l'évaluation.

IMPORTANT : usage des outils d'IA générative

Le candidat doit être capable d'expliquer et de justifier son code, de répondre aux questions relatives à la méthodologie, aux choix effectuées, à la performance des modèles obtenus, ainsi qu'aux limites et pistes d'amélioration. Il devra démontrer une **compréhension réelle des solutions proposées**.

Une utilisation excessive ou non maîtrisée des outils d'IA générative sera pénalisée.

L'équipe DATALAB reste disponible durant la période du projet pour répondre aux questions techniques et méthodologiques.

3 Contexte, données et méthodologie

Vous travaillez en tant qu'ingénieur IA pour une entreprise spécialisée dans l'analyse et l'exploitation des données multimodales (images, textes et séries temporelles) afin d'optimiser la prise de décision et d'améliorer les performances de ses services.

Dans ce contexte, votre mission consiste à concevoir, implémenter et documenter des pipelines de modèles d'apprentissage automatique adaptés aux différents types de données traités par l'entreprise.

Dans ce cadre, vous intervenez sur plusieurs problématiques techniques couvrant **l'ensemble du cycle de vie des modèles d'IA** :

- Développer un pipeline de classification d'images à l'aide de réseaux de neurones profonds.
- Mettre en place un pipeline de classification de textes en combinant des méthodes traditionnelles de NLP et des modèles de réseaux de neurones, notamment LSTM.
- Concevoir et implémenter une approche de fusion multimodale combinant plusieurs types de données (images et textes) pour améliorer les performances des modèles d'IA.
- Développer un pipeline de prédiction basé sur des séries temporelles.
- Assurer la gestion des expérimentations et le versionnement des modèles.
- Implémenter une API permettant d'exposer le pipeline d'inférence complet d'un modèle ML, incluant le prétraitement des données et la génération des prédictions.
- Déployer un pipeline MLOps intégrant CI/CD afin d'automatiser le cycle de vie des modèles développés.
- Mettre en place un monitoring du pipeline d'inférence en production afin de suivre l'utilisation, les performances et le comportement du modèle en production.

Dans le cadre de cette mise en situation professionnelle reconstituée, il vous est demandé de réaliser plusieurs sous-projets dont les objectifs, les données et les méthodologies sont détaillés dans les chapitres qui suivent.

3.1 Classification avec des données multimodales

L'objectif est d'explorer la fusion de données multimodales en combinant des images et leurs descriptions textuelles afin d'améliorer la classification. Dans un premier temps, des modèles unimodaux seront entraînés séparément sur les informations visuelles et textuelles. Dans un second temps, une approche multimodale sera conçue pour combiner ces deux sources d'information, afin d'améliorer les performances de classification et d'enrichir l'interprétation des résultats.

Modules concernés

- M24 Deep Learning II,
- M32 Machine Learning II (explicabilité des modèles).

Données

Vous aurez à votre disposition un set de données multimodales (images et textes) disponible sur Kaggle (Linwei Tao. Multi-label Classification Competition 2023). Ce dataset est conçu pour un problème de **classification multi-label**, où chaque image peut être associée à plusieurs labels.

Description du dataset

data/*.jpg : un ensemble d'images qui couvrent une variété de scènes et d'objets.

train.csv et test.csv : fichiers contenant des métadonnées des images :

- **ImageID** (nom du fichier) : Chaque ligne du fichier CSV correspond à une image spécifique.
- **Caption** : Chaque image est accompagnée d'une courte description en texte libre. Ces descriptions fournissent des informations contextuelles sur l'image, telles que le contenu principal, la scène ou l'environnement représenté.
- **Labels** : Chaque image est annotée avec un ou plusieurs labels indiquant les catégories auxquelles elle appartient. Le fichier test.csv ne contient pas de labels. Les labels sont représentés par des nombres entiers compris entre 1 et 19, à l'exception de 12, ce qui correspond à **18 classes distinctes**.

Exemple de données :

Image	Texte descriptif (caption)	Labels
	Woman in swim suit holding parasol on sunny day.	1

Variable cible

Le dataset présente des labels sous forme numérique, ce qui signifie qu'ils ne sont pas directement interprétables sans analyse supplémentaire. Cependant, avec une exploration rapide des données, on peut observer que certains labels apparaissent systématiquement dans des contextes similaires, suggérant qu'ils correspondent à des catégories spécifiques d'images.

Voici quelques exemples d'images annotées avec le label « 5 » :

Labels	Caption
5	A big *airplane* flying in the big blue sky.
3, 5	A small *airplane* that is on a runway.
3, 5, 6, 8, 10	An *airplane* sitting in the middle of a city.
1, 5	People gather around as a man gets into an old-time *airplane*.
1, 5	A jet sits on display in during an *air show*.

Cette observation suggère que le label 5 est probablement lié à la catégorie "*avion*". Une analyse plus approfondie, telle que la visualisation d'autres images associées à ce label, confirmerait cette hypothèse et permettrait de mieux interpréter les autres labels du dataset.

Le modèle final doit être capable de prédire, pour une image donnée, le(les) label(s) correspondant(s) en utilisant une approche **unimodale** (en se basant uniquement sur les images ou les textes descriptifs), ou bien une approche **multimodale**, en combinant les deux types de données. La méthodologie de travail est détaillée dans les chapitres 2.1.1-2.1.4.

3.1.1 Exploration des données

L'objectif est de se familiariser avec les données multimodales, comprendre la structure du dataset et identifier les défis liés à la classification multi-label avant de procéder à l'entraînement du modèle.

MÉTHODOLOGIE

- Analyse générale du dataset : examiner la taille du dataset et détecter d'éventuelles données manquantes.
- Analyse de la variable cible : identifier le nombre total de labels uniques ; analyser la distribution des labels et afficher un histogramme des occurrences ; établir une statégie pour gérer le déséquilibre des classes.
- Exploration des images : charger et afficher quelques images aléatoires du dataset.
- Analyse des descriptions textuelles : afficher quelques exemples de descriptions associées aux images, analyser la longueur moyenne des descriptions (nombre de mots), identifier les mots les plus fréquents à l'aide d'un word cloud.
- Exploration des relations entre les images, les textes et les labels : identifier d'éventuels patterns entre certains labels et leurs images/descriptions.
- Séparation des données : 70% train, 15% validation, 15% test. Utilisez uniquement le fichier «train.csv» contenant les données annotées.

3.1.2 Classification des images

L'objectif est de développer un modèle de classification multi-label basé uniquement sur les images.

MÉTHODOLOGIE

- **Préparation des données**
  - Charger les données.
  - Prétraiter les images : redimensionner et normaliser les pixels ([0,1] ou [-1,1]).
- **Expérimentation de modèles CNN**
  Vous devrez concevoir une stratégie expérimentale permettant de développer et comparer plusieurs architectures de classification multi-label basées sur des CNN.
  - Définir un protocole d'évaluation adapté à la classification multi-label, en tenant compte les résultats de l'analyse exploratoire : choix des métriques, stratégie de validation et analyse des erreurs. Pensez à y inclure la visualisation des courbes d'apprentissage (loss et métriques) pour diagnostiquer le sous-apprentissage/sur-apprentissage.
  - Concevoir une première architecture CNN simple servant de baseline.
  - Proposer et mettre en œuvre quelques stratégies d'amélioration parmi celles vues en cours, par exemple utilisation d'architectures CNN pré-entraînées via transfer learning (feature extractor) ou fine-tuning partiel.
  - Documenter et analyser les performances de chaque modèle, afficher les métriques choisies, ainsi que les matrices de confusion (classe par classe), et interpréter les résultats.
- **Comparaison des résultats et choix du modèle final**
  - Comparer les performances des modèles expérimentés en respectant le protocole d'évaluation défini précédemment.
  - Justifier le choix du modèle retenu.
- **Analyse du modèle final**
  - Afficher quelques images de test avec les prédictions et les scores de confiance correspondants.
  - Utiliser des techniques de XAI, comme LIME, pour expliquer les décisions du modèle.
  - Examiner des erreurs de prédiction et essayer d'identifier des explications.
  - Interpréter les résultats et proposer des pistes d'amélioration.

3.1.3 Classification des textes

L'objectif est de développer un modèle de classification multi-label basé uniquement sur les textes descriptifs des images.

MÉTHODOLOGIE

- **Préparation des données**
  - Définir un pipeline de prétraitement des textes (variable « Caption »), en tenant compte les résultats de l'analyse exploratoire et en appliquant les techniques étudiées dans le cours.
  - Appliquer la même répartition que pour les images (70% train, 15% validation, 15% test) afin de garantir l'alignement des modalités.
- **Expérimentations de modèles NLP**
  Vous devrez concevoir une stratégie expérimentale permettant de comparer plusieurs approches de classification multi-label sur des données textuelles.
  - Définir un protocole d'évaluation cohérent avec celui utilisé pour les images.
  - Réaliser un modèle baseline avec un algorithme de Machine Learning classique avec un encodage des textes adapté.

  - Proposer et mettre en œuvre quelques stratégies d'amélioration parmi celles vues en cours, en faisant varier la représentation des textes et les architectures deep learning adaptées aux séquences textuelles, par exemple LSTM, Bi-LSTM ou CNN 1D.
  - Documenter et analyser les performances de chaque modèle, afficher les métriques choisies, ainsi que les matrices de confusion (classe par classe), et interpréter les résultats.
- **Comparaison des résultats et choix du modèle final**
  - Comparer les performances des modèles expérimentés en respectant le protocole d'évaluation défini précédemment.
  - Justifier le choix du modèle retenu.
- **Analyse des résultats**
  - Afficher quelques textes de test avec les prédictions et les scores de confiance correspondants.
  - Utiliser des techniques de XAI, comme LIME, pour expliquer les décisions du modèle.
  - Examiner des erreurs de prédiction et essayer d'identifier des explications.
  - Interpréter les résultats et proposer des pistes d'amélioration.

3.1.4 Fusion multimodale

L'objectif est de développer une approche multimodale combinant texte et images afin d'améliorer la classification multi-label, en s'appuyant sur les résultats obtenus dans les parties unimodales.

MÉTHODOLOGIE

- **Conception de la stratégie multimodale**
  A partir des modèles unimodaux image et texte retenus précédemment, vous devrez définir et mettre en œuvre une ou plusieurs approches de fusion multimodale.

  Votre démarche devra inclure au minimum deux types de fusion étudiés en cours.

  Le choix des architectures, des stratégies de validation et des hyperparamètres reste ouvert et doit être justifié.

  - **Early fusion (feature-level fusion)** : combiner des représentations unimodales préalablement extraites afin d'entraîner un modèle de classification commun.
  - **Joint fusion (apprentissage end-to-end)** : concevoir une architecture intégrant simultanément les deux modalités et permettant un apprentissage conjoint des représentations et de la décision finale.
- **Analyse comparative des solutions**
  - Comparer les performances des modèles multimodaux avec celles des modèles unimodaux (images seules, textes seuls).
  - Comparer les contributions respectives des modalités texte et image dans la décision finale.
  - Justifier le choix du modèle retenu.

3.2 Prévision des séries temporelles

L'objectif est de concevoir un pipeline complet de prévision de séries temporelles, allant de l'analyse exploratoire à la mise en production et le monitoring.

Modules concernés

- M32 Machine Learning II
- M34 MLOps
- M27 DevOps

Données

Vous utiliserez des données météo historiques fournies par Historical Weather API (Open-Meteo). Cette API repose sur des ensembles de données de réanalyse des observations provenant de différentes sources (stations météorologiques, radars, satellites, etc.) et, grâce à des modèles mathématiques, fournit des informations météorologiques historiques détaillées, y compris pour des zones sans station météorologique à proximité.

Vous devez récupérer une **série temporelle des températures à 2m du sol** (« Temperature (2 m) ») pour une ville de votre choix, en utilisant ses coordonnées géographiques (latitude, longitude).

Objectif

Développer un modèle capable de prédire la température dans la ville de votre choix pour un horizon de 3 jours, avec un pas de temps de 3h (00h, 03h, 06h, ..., 21h).

Exemple de données

Time	temperature_2m (°C)
2022-01-01T00:00	27.2
2022-01-01T01:00	26.8
2022-01-01T02:00	27.0
2022-01-01T03:00	27.0
2022-01-01T04:00	27.1
2022-01-01T05:00	27.2
2022-01-01T06:00	27.2
2022-01-01T07:00	27.2
2022-01-01T08:00	27.2
2022-01-01T09:00	27.1
2022-01-01T10:00	27.1
2022-01-01T11:00	27.2
2022-01-01T12:00	27.1

MÉTHODOLOGIE

- **Acquisition et préparation des données**
  - Définir la période d'étude cohérente avec vos expérimentations et justifier ce choix.
  - Agréger les données pour obtenir une nouvelle série avec un pas de temps de 3 heures : la valeur à 00h sera la moyenne des valeurs mesurées à 00h, 01h et 02h ; la valeur à 03h sera la moyenne des valeurs mesurées à 03h, 04h et 05h ; etc.
  - Utiliser cette série transformée dans l'ensemble des expérimentations.
- **Analyse exploratoire**
  - Visualiser la série temporelle et identifier d'éventuelles anomalies ou patterns récurrents.
  - Analyser la tendance, la saisonnalité, les résidus, l'autocorrelation.
  - Formuler des hypothèses sur la dynamique de la série et leurs implications pour la modélisation.
  - Documentez les résultats de manière synthétique.
- **Stratégie expérimentale**
  Il n'est pas attendu d'implémenter toutes les architectures vues en cours. Le choix d'un nombre très limité de modèles pertinents, correctement analysés et comparés, sera valorisé.
  - Définir un protocole d'évaluation cohérent avec une séparation temporelle train / validation / test, des métriques adaptées, une analyse des erreurs.
  - Mettre en place une approche statistique simple servant de baseline.

- Expérimenter au moins une architecture vue en cours, par exemple CNN temporels, modèles MLP modernes, modèles linéaires modernes (ex. DLinear).

- **Déploiement du modèle**
  - Implémenter le pipeline d'entraînement, résultant de la phase d'expérimentation, dans un script Python : préparation de données, entraînement, évaluation, sauvegarde du modèles et d'autres artefacts si nécessaire.
  - Créer une base de données SQLite contenant :
    - Les données météo historiques,
    - Les prédictions générées : identifiant du modèle utilisé, date, horizon, prédictions.
  - Mettre en place une approche batch pour la génération des prédictions. Les prédictions sont calculées périodiquement, stockées dans une base de données et accessibles via une API. **Dans le cadre de ce projet, l'orchestration automatique du réentraînement n'est pas obligatoire, mais le workflow doit pouvoir être exécuté de manière reproductible.**
    - Télécharger les données récentes, par rapport à une date donnée, via l'API et les sauvegarder dans la base de données,
    - Entraîner un nouveau modèle en utilisant votre pipeline d'entraînement sur une période mise à jour,
    - Sauvegarder le modèle, les metadonnées pertinentes et d'autres artefactes, si nécessaire, dans Mlflow (votre registre de modèles),
    - Générer des prédictions pour une période prédéfinie,
    - Stocker les prédictions dans la base de données.
  - Réaliser une API REST avec FastAPI pour accéder aux prédictions.
  - Mettre en place des tests unitaires et d'intégration.
  - Mettre en place un logging pertinent (requêtes, versions de modèle, erreurs, latences).
  - Mettre en œuvre un endpoint de version (version logicielle et version modèle).
  - Mettre en œuvre un monitoring minimal :
    - suivi des erreurs (lorsque les observations réelles sont disponibles),
    - visualisation simple (tableau ou graphique).
  - Réaliser un pipeline CI/CD basé sur GitHub Actions.

4 Livrables

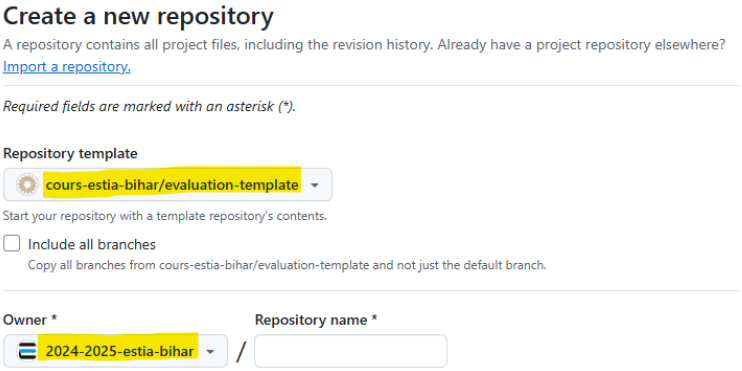
Vous devez créer un dépôt github unique pour la totalité des livrables du projet.

La documentation doit être faite en anglais ou en français.

Bonnes pratiques :
- Créer un repository github dans l'organisation dédiée (les informations seront envoyées séparément) dès le début du projet et pousser le code graduellement,
- Utiliser le template du projet (cf. chapitre 4.1),
- Respecter des conventions de nommage des dossiers et des fichiers.

4.1 Structure du dépôt github

Créer un repository github dans l'organisation dédiée en utilisant le template suivant : https://github.com/cours-estia-bihar/evaluation-template.



Voici les détails de la structure :

- **notebooks/**
  - Analyse exploratoire des données multimodales
  - Classification d'images
  - Classification de textes
  - Classification multimodale
  - Prévision des séries temporelles
- **data/**
  - Base de données SQLite contenant des données d'entraînement des modèles et des prédictions générées.
- **model/**
  - registry/ : registre des modèles entraînés.
- **monitoring/**
  - output/ (dossier contenant les images générées)
- **src/**
  - common/
  - data/ : acquisition des données récentes manquantes, en fonction d'une date donnée (fin de période).
  - training/ : pipeline d'entraînement
  - inference/ : génération des prédictions à une date donnée (batch).
  - monitoring/ : génération d'un graphique comparant les prédictions avec les données observées
- **api/**
  - main.py : implémentation des endpoints pour les fonctionnalités suivantes :
    - Récupération des prédictions pour une date donnée
    - Récupération des prédictions combinées avec des données réelles observées pour une période donnée
    - Récupération de la version logicielle et version modèle actuelles. Pour la version logicielle, retourne 0.0.0 en local et le commit ID quand compilé depuis le CI/CD.
- **README.md**
  - Description du projet, de l'architecture et des flux de données
  - Instructions pour l'installation des dépendances, run, build et test
  - Description du pipeline CI/CD
  - **Note : un README générique, superficiel ou manifestement généré automatiquement sera pénalisé.**

4.2 Structure des notebooks Jupyter

Utilisez les cellules markdown pour structurer vos notebooks de façon suivante :

- Description synthétique du projet et de ses objectifs
- Chargement des données
- Analyse exploratoire des données
- Séparation des données en ensembles d'entraînement, de validation et de test
- Prétraitement des données
- Modélisation et évaluation des performances des modèles
- Analyse et interprétation des résultats obtenus

Les notebooks doivent inclure les résultats des cellules exécutées. Aucun notebook ne sera réexécuté par les examinateurs.
Veillez cependant à **ne pas inclure les sorties trop volumineuses** (images trop lourdes, tensors, logs détaillés, etc.), qui alourdiraient inutilement le dépôt. Les impressions temporaires utilisées pour le débogage ou la compréhension personnelle ne doivent pas apparaître dans la version finale.

4.3 Support de présentation

Pour chaque sous-projet ML/DL, l'apprenant devra présenter :

- Les spécifications, les objectifs et les métriques d'évaluation de performance choisies,
- Les résultats synthétiques de l'analyse exploratoire des données,
- La description synthétique du prétraitement réalisé et la justification des choix faits,
- La description synthétique des expérimentations menées et leur justification : algorithmes et leurs paramètres, architectures,
- Les résultats obtenus, leur comparaison et interprétation.

Pour la partie MLOps/DevOps, l'apprenant devra présenter :

- L'architecture de l'application et le flux des données
- Le pipeline de CI/CD avec la description des outputs de ses différentes étapes

Le travail fourni doit être soigneusement relu et maitrisé. L'accent sera donné à la capacité à expliquer les différents aspects techniques et points durs rencontrés.

La **démonstration** concerne uniquement l'API exposant le modèle ML requis. L'apprenant devra préparer un **scénario** qui permet de montrer le fonctionnement des différents endpoints de l'API dans le créneau imparti.

5 Modalités d'évaluation

Pour l'ensemble des modèles produits, **le suivi de la méthodologie et la justification des différents choix effectués seront prioritaires dans l'évaluation**, par rapport à la qualité des prédictions obtenues.

Les analyses et conclusions doivent s'appuyer sur des éléments mesurables. Les formulations vagues, telles que « le modèle ne prédit pas bien » ou « l'amélioration n'est pas significative », ne seront pas considérées comme suffisantes sans indicateurs quantitatifs (métriques, comparaisons expérimentales, visualisations ou analyses d'erreurs).

M32 MACHINE LEARNING II, **M33 DEEP LEARNING II** :

- Les notebooks Jupyter respectent la structure demandée (voir le chapitre correspondant).
- La méthodologie est respectée (voir le chapitre correspondant à chaque sous-projet).
- Les blocs de codes sont commentés. Les commentaires peuvent être succincts mais ils doivent permettre de comprendre la logique et l'organisation de votre code.
- Les graphiques doivent avoir des titres, des noms corrects des axes, des légendes et des commentaires textuels.
- Les choix doivent être justifiés.
- Transformations des données requises par les algorithmes utilisés, sont faites.
- Il ne doit pas y avoir de fuite des données.
- Les principaux résultats doivent être résumés et expliqués.
- Les résultats doivent être reproductibles.

M27 DEVOPS

- *Compétence : Créer une image docker pour empaqueter le code Python de l'API incluant des fichiers nécessaires (tels que modèles ML sérialisés par exemple). Références des séquences de cours : devops_3_docker, devops_5_deploy.*

  Critères d'évaluation :
  - Le conteneur démarre et sert les endpoints.
  - Les couches sont optimisées pour ne pas réinstaller les dépendances lorsque le code change.
  - La documentation sur la façon de générer l'image et l'architecture est fournie.

- *Compétence : Ajouter un logging pertinent pour chaque handler de endpoint. Ajouter un endpoint pour connaitre la version logicielle actuelle. Références de la séquence de cours : devops_6_monitor_operate.*

  Critères d'évaluation :
  - Une ligne de log dans les handlers de endpoint.
  - Le logging systématique des erreurs.
  - Le niveau de log approprié
  - Un endpoint GET /version qui fonctionne et retourne la version du logiciel (string).
  - La documentation sur le endpoint GET /version.

- *Compétence : Concevoir, implémenter et exécuter des tests automatisés des endpoints d'une API. Références de la séquence de cours : devops_4_testing_strategies.*

  Critères d'évaluation :
  - Les tests passent et permettent d'effectuer des contrôles pertinents.
  - La documentation sur la façon d'exécuter les tests localement.

- *Compétence : Définir un pipeline basé sur GitHub Actions qui crée une image docker, l'envoie vers ghcr.io et exécute des tests de l'API. Références de la séquence de cours : devops_7_automate.*

  Critères d'évaluation :
  - Les dépendances des jobs sont correctement définies.
  - La création et l'envoi des images docker vers GHCR.
  - L'exécution des tests de l'API pour la dernière image docker créée.
  - Le pipeline est opérationnel et ne contient pas de secrets en texte brut dans le code.